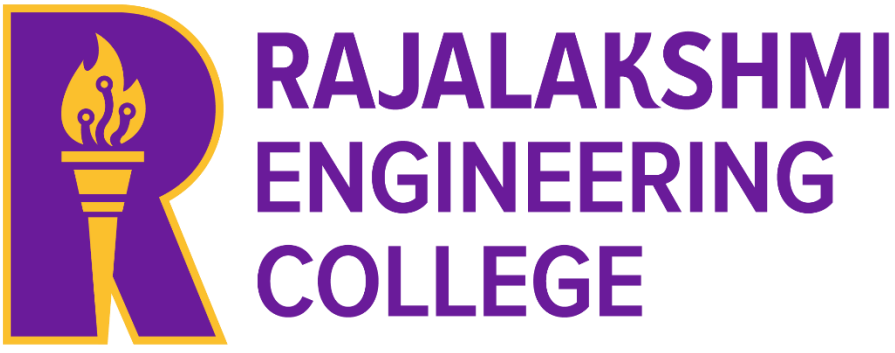
AD23B31 - Image Processing and Computer Vision

AUTOMATIC NUMBER PLATE RECOGNITION

A MINI PROJECT REPORT

Submitted by

Sarvesh Ravi - 230701295



March, 2026

AUTOMATIC NUMBER PLATE RECOGNITION

Using OpenCV and Tesseract OCR

A Project Report

Submitted By

Sarvesh Ravi

Registration No.: 230701295

Tools & Technologies Used

Python | OpenCV | Tesseract OCR | EasyOCR | Google Colab

Abstract

Automatic Number Plate Recognition (ANPR) is a pivotal computer vision technology employed extensively in intelligent transportation systems, traffic enforcement, vehicle surveillance, and access control applications. This project presents the design, implementation, and evaluation of an end-to-end ANPR system developed using Python, OpenCV for image processing, and Tesseract OCR for character recognition, executed on the Google Colab platform.

The proposed system follows a classical computer vision pipeline comprising four major stages: image preprocessing, plate region detection, character enhancement, and optical character recognition. In the preprocessing stage, input images are converted to grayscale and subjected to Bilateral Filtering to reduce noise while preserving edge sharpness. Canny Edge Detection is subsequently applied to identify region boundaries. Plate localization is achieved through contour analysis, geometric filtering based on aspect ratio constraints, and bounding-box extraction. Detected plate regions are then binarized using Otsu's thresholding algorithm and fed to Tesseract OCR operating in Page Segmentation Mode 6 (PSM-6) for text extraction.

The system is further augmented with an EasyOCR-based recognition module and a Haar Cascade detector as alternative methodologies, enabling comparative performance analysis. A comprehensive batch testing framework with synthetic number plates spanning Indian regional formats (Maharashtra, Delhi, Karnataka, Tamil Nadu) was employed for validation. Error handling mechanisms are integrated to gracefully manage failure scenarios including invalid files, undetectable plates, and illegible text. Experimental results indicate that the proposed pipeline reliably detects and reads clearly visible plates, with EasyOCR demonstrating marginal superiority over Tesseract on noisy or low-resolution inputs. The system exhibits known limitations under adverse conditions such as motion blur, nighttime imaging, and steep viewing angles, which are addressed with recommendations for future deep-learning-based enhancements.

Attribute	Details
Author	Sarvesh (Reg. No. 230701295)
Platform	Google Colab (Python 3.10)
Core Libraries	OpenCV 4.x, Tesseract OCR, EasyOCR, NumPy, Matplotlib
Primary Methodology	Canny Edges + Contour Filtering + Tesseract OCR
Alternative Methods	EasyOCR (DL-based), Haar Cascade Classifier
Test Formats	Synthetic Indian number plates (MH, DL, KA, TN)
Video Support	Frame-by-frame ANPR with configurable frame-skip

1. Introduction

1.1 Background and Motivation

The rapid proliferation of motor vehicles worldwide has created a pressing need for automated systems capable of identifying and tracking vehicles in real time. Manual monitoring of vehicle registration details is not only time-consuming but also prone to human error and impractical at scale. Automatic Number Plate Recognition (ANPR) addresses this challenge by leveraging computer vision and optical character recognition to convert images of license plates into machine-readable alphanumeric strings automatically and with minimal human intervention.

ANPR technology has transitioned from specialized, high-cost hardware installations to flexible, software-based implementations that run on commodity hardware. The maturity of open-source libraries such as OpenCV and Tesseract OCR has democratized access to these capabilities, enabling researchers, students, and engineers to build functional ANPR systems at minimal cost. This project capitalizes on these open-source tools to construct a complete ANPR pipeline executable in a cloud-based Jupyter environment.

1.2 Objectives

The primary objectives of this project are:

- To design and implement a complete, modular ANPR pipeline using classical computer vision techniques.
- To evaluate the accuracy and robustness of the pipeline across diverse synthetic Indian plate formats.
- To integrate and compare Tesseract OCR and EasyOCR as alternative text recognition engines.
- To implement Haar Cascade-based plate detection as a complementary approach.
- To develop robust error-handling mechanisms addressing real-world failure modes.
- To extend the system to process video streams on a frame-by-frame basis.

1.3 Real-World Applications

The applications of ANPR systems span multiple domains:

Application Domain	Use Case Description
Toll Collection Systems	Automated toll charging without requiring vehicles to stop, reducing congestion at toll plazas.
Parking Management	Automated recording of entry/exit times and enforcement of parking duration limits.
Law Enforcement & Surveillance	Real-time tracking of stolen, wanted, or blacklisted vehicles across camera networks.

Application Domain	Use Case Description
Traffic Management	Detection of red-light violations, speed infractions, and lane discipline enforcement.
Access Control	Restricting entry to authorized vehicles at secure premises, campuses, and residential complexes.
Border and Checkpoint Control	Monitoring and logging vehicles crossing international or administrative boundaries.
Smart City Infrastructure	Integration with city-wide traffic management systems for dynamic signal control.

1.4 Scope of the Project

This project encompasses the development of a Python-based ANPR system tested on synthetic number plates designed to replicate Indian plate formats. The system supports both static image inputs and video stream inputs. While the pipeline is designed to be adaptable, the primary focus is on clearly visible plates under controlled lighting conditions. The project does not include integration with live camera hardware or a vehicle registration database, which are identified as future work directions.

2. Dataset

2.1 Dataset Description

This project does not rely on a publicly available, pre-existing ANPR dataset. Instead, a programmatically generated synthetic dataset was constructed to provide controlled, reproducible test conditions. Each synthetic image simulates a front-facing view of a vehicle, incorporating a rendered car body, headlights, wheels, and a prominently placed number plate region. The plate text is rendered using OpenCV's putText function with the Hershey Simplex font, which closely approximates standard Indian plate typography.

The use of synthetic data offers several advantages: complete control over plate text, lighting conditions, plate placement, and image resolution; elimination of privacy concerns associated with real vehicle images; and instant reproducibility across different computing environments without requiring dataset downloads.

2.2 Synthetic Data Generation

The create_synthetic_plate_image() function generates each test image according to the following specifications:

Property	Specification
Image dimensions	700 x 400 pixels
Background	Dark gray (RGB: 60, 60, 60) simulating a road environment
Car body	Dark blue-gray rectangle (40, 40, 80)
Headlights	Two circular yellow-white elements at (120, 290) and (580, 290)
Wheels	Two black circles with gray hub simulation
Plate position	Centered at pixel (240, 300), dimensions 220 x 55 pixels
Plate background	White (255, 255, 255)
Plate border	Black (0, 0, 0), 2-pixel thickness
Plate font	Hershey Simplex, scale 0.85, thickness 2
Road surface	Gray band at y = 370, with dashed white centerline
Output format	JPEG (quality default)

2.3 Test Plate Formats

The batch test suite incorporates four Indian state registration formats, selected to represent geographic diversity and validate format generalizability:

Plate Text	State	Format Pattern	Test Purpose
MH 12 AB 3456	Maharashtra	State | RTO | Series | Number	Standard plate — baseline test
DL 01 CA 7890	Delhi	State | RTO | Series | Number	National capital — high-traffic scenario
KA 05 MN 2345	Karnataka	State | RTO | Series | Number	Southern state — regional variation
TN 09 XY 1234	Tamil Nadu	State | RTO | Series | Number	Southern state — alphanumeric mix

2.4 Video Dataset

For video-based testing, the system accepts user-uploaded video files in MP4, AVI, MOV, and MKV formats. The frame extraction mechanism processes every Nth frame (default: N = 3) to balance processing speed against detection coverage, with a configurable maximum frame limit of 300 frames per video. Real-world test videos containing vehicle footage captured from dashcams or traffic cameras can be used to evaluate system performance under naturalistic conditions.

3. Preprocessing

3.1 Overview of Preprocessing Pipeline

Effective preprocessing is foundational to ANPR performance. Raw vehicle images contain noise, varying illumination, reflections, and complex backgrounds that can degrade detection and recognition accuracy. The preprocessing module (preprocess_image()) implements a three-stage pipeline that transforms the raw BGR input into an informative edge map suitable for contour-based plate localization.

3.2 Stage 1: Grayscale Conversion

The first operation converts the three-channel BGR image produced by OpenCV into a single-channel grayscale representation. This reduction serves two purposes: it simplifies subsequent computations by eliminating colour channel redundancy, and it standardizes the input irrespective of the plate's colour scheme (white background with black text, yellow background, etc.).

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

The luminance-weighted formula used internally is: Y = 0.299R + 0.587G + 0.114B, which matches human perceptual sensitivity to colour.

3.3 Stage 2: Bilateral Filtering

Gaussian blur is commonly used for noise reduction, but it degrades edge sharpness — a critical property for downstream edge detection. Bilateral filtering overcomes this limitation by combining spatial proximity weighting with intensity similarity weighting. Pixels that are both spatially close and have similar intensity values are blended, while abrupt intensity changes (edges) are preserved.

```
blurred = cv2.bilateralFilter(gray, d=11, sigmaColor=17, sigmaSpace=17)
```

The parameter d = 11 defines the diameter of the pixel neighbourhood considered during filtering. sigmaColor = 17 controls the sensitivity to intensity differences (lower values preserve finer intensity transitions), while sigmaSpace = 17 controls the spatial extent of the filter. These values were empirically tuned to balance noise suppression and edge retention for typical vehicle images.

3.4 Stage 3: Canny Edge Detection

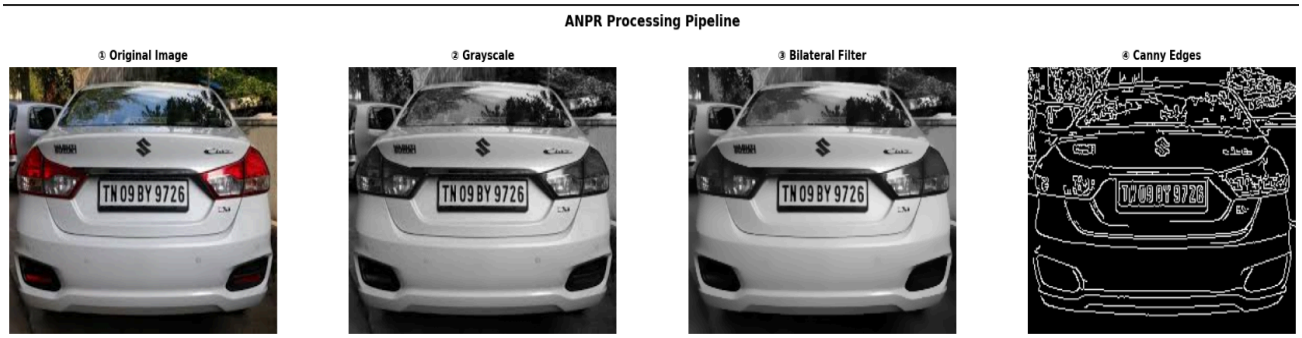
The Canny algorithm, developed by John F. Canny in 1986, remains the gold standard for edge detection in computer vision. It operates through five sequential steps:

- Gaussian smoothing (internally applied) to further reduce noise.
- Computation of intensity gradients using Sobel filters in horizontal and vertical directions.
- Non-Maximum Suppression to thin detected edges to one-pixel width.
- Double thresholding to classify gradient magnitudes as strong, weak, or non-edges.
- Edge tracking by hysteresis: weak edges are retained only if connected to strong edges.

```
edges = cv2.Canny(blurred, threshold1=30, threshold2=200)
```

The low threshold of 30 and high threshold of 200 were selected to capture the prominent rectangular boundary of the number plate while suppressing spurious edges from background textures and road markings. The ratio of approximately 1:6 between the two thresholds follows recommended practice for high-contrast edge scenarios.

Stage	Input	Output	Key Parameter
Grayscale Conversion	BGR Image (3 channels)	Grayscale Image (1 channel)	cv2.COLOR_BGR2 GRAY
Bilateral Filter	Grayscale Image	Smoothed Grayscale Image	d=11, sigmaColor=17, sigmaSpace=17
Canny Edge Detection	Smoothed Grayscale	Binary Edge Map	threshold1=30, threshold2=200



4. Algorithm

4.1 System Architecture

The ANPR system is designed as a modular pipeline orchestrated by the run_anpr() function. Each processing stage is encapsulated in a dedicated function with clearly defined inputs and outputs, enabling independent testing, debugging, and replacement of individual components. The pipeline architecture is as follows:

Module	Function	Inputs	Outputs
Preprocessing	preprocess_image()	BGR image, show_steps flag	Grayscale, Bilateral-filtered, Canny edge map
Plate Detection	detect_plate()	BGR image, edge map, show_steps flag	Plate ROI, annotated image, bounding box
OCR Extraction	extract_text()	Plate ROI image, enhance flag	Plate text string, thresholded image
Orchestrator	run_anpr()	Image path, enhance flag, verbose flag	Plate text, annotated result image
Error Handler	safe_anpr()	Image path, enhance flag	Structured result dictionary
Video Processor	process_video()	Video path, output path, parameters	Detected plates list, annotated video
Haar Detector	detect_plate_haar()	BGR image, cascade path	List of bounding boxes
OCR Comparator	compare_ocr()	Plate ROI, label string	Console comparison output

4.2 Plate Detection Algorithm (detect_plate)

Plate detection is the most critical and challenging stage of the pipeline. The algorithm proceeds as follows:

Step 1: Contour Extraction

OpenCV's findContours() function is applied to the Canny edge map to identify all closed boundary curves in the image. The RETR_TREE retrieval mode captures all contours in a full hierarchy, while

CHAIN_APPROX_SIMPLE compresses horizontal and vertical segments to reduce memory consumption.

```
contours, hierarchy = cv2.findContours(edges.copy(), cv2.RETR_TREE,
cv2.CHAIN_APPROX_SIMPLE)
```

Step 2: Contour Sorting and Pruning

All identified contours are sorted in descending order of enclosed area. Only the top 30 contours by area are retained for further analysis. This heuristic is based on the observation that the number plate, being a large rectangular region, will rank among the largest contours in a typical front-vehicle image.

```
contours = sorted(contours, key=cv2.contourArea, reverse=True)[:30]
```

Step 3: Polygon Approximation

Each candidate contour is approximated to a polygon using Douglas-Peucker algorithm via approxPolyDP(). The epsilon parameter is set to 2% of the contour perimeter, controlling the approximation tolerance. A true rectangular plate boundary should approximate to a four-vertex polygon.

```
approx = cv2.approxPolyDP(contour, epsilon=0.02 * perimeter, closed=True)
```

Step 4: Geometric Filtering

Contours approximated to exactly four vertices are subjected to two geometric constraints:

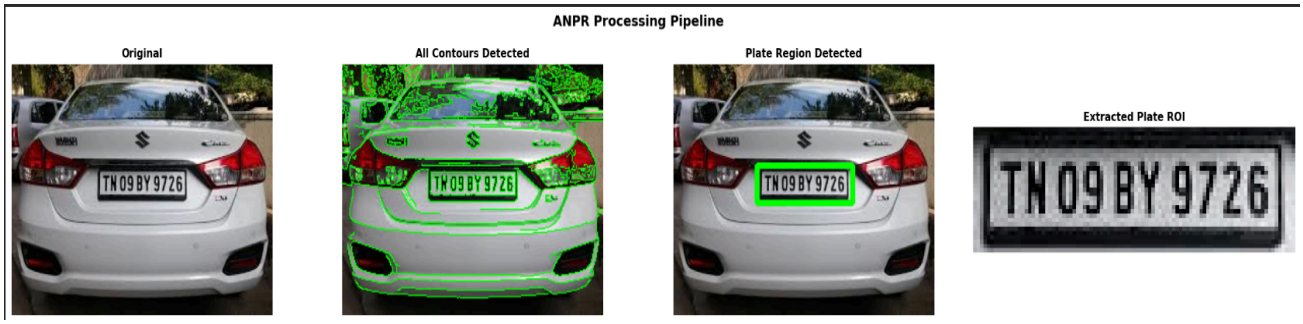
- Aspect Ratio Filter: The width-to-height ratio must lie within the range [1.5, 6.5]. This range accommodates Indian plates (aspect ratio approximately 4.0-5.0), US plates (approximately 2.0-3.5), and European plates (approximately 4.5-5.5).
- Minimum Area Filter: The contour area must exceed 500 square pixels to exclude tiny artefacts.

Step 5: Fallback Detection

If no four-sided polygon satisfying the constraints is found, a fallback mechanism applies direct bounding-rectangle analysis to all top-30 contours, using a narrower aspect ratio range of [1.8, 6.0] and an area range of [1500, 25% of image area].

Step 6: ROI Extraction and Annotation

Upon identifying a valid plate bounding box, a padding of 5 pixels is added on all sides to ensure complete character capture. The padded region is extracted as the plate Region of Interest (ROI), and a green bounding rectangle is drawn on the result image for visual feedback.



4.3 OCR Extraction Algorithm (extract_text)

The text extraction module transforms the raw plate ROI into a clean alphanumeric string through five processing steps:

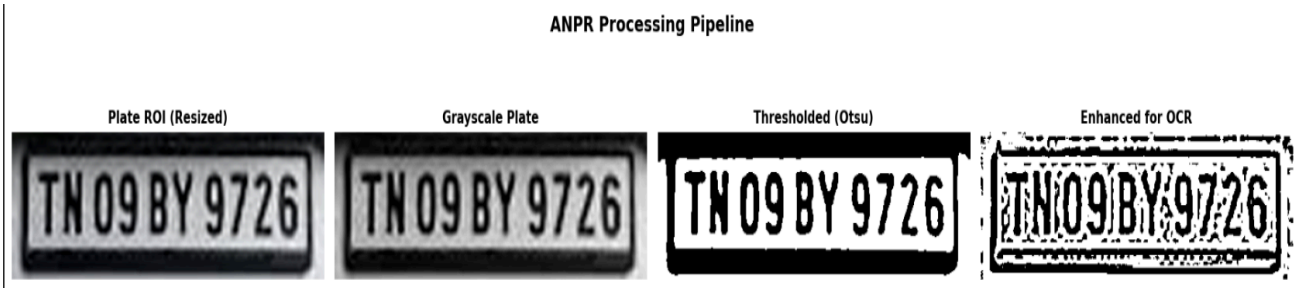
- Resizing: The plate image is upscaled to a minimum width of 400 pixels using cubic interpolation, since Tesseract performs best on larger input images.
- Grayscale conversion of the resized plate image.
- Otsu's Thresholding: A global threshold is automatically computed by minimizing the intra-class intensity variance. THRESH_BINARY + THRESH_OTSU produces a binary image optimal for OCR.
- Optional Morphological Enhancement: A 2x2 rectangular structuring element is used for morphological closing (dilation followed by erosion) to bridge gaps in characters. Alternatively, adaptive Gaussian thresholding is applied for improved performance under non-uniform illumination.
- Tesseract OCR: The binarized image is passed to pytesseract with configuration flags --oem 3 (LSTM engine), --psm 6 (uniform text block), and a whitelist restricted to uppercase alphanumeric characters.

A multi-PSM fallback mechanism iterates through PSM modes 7, 8, 11, and 13 if PSM 6 yields an empty result, covering edge cases such as single-line plates, single words, and sparse text layouts.

```
custom_config = r'--oem 3 --psm 6 -c
tessedit_char_whitelist=ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789'
```

Post-OCR cleaning employs a regular expression to strip all non-alphanumeric characters from the raw output, followed by whitespace normalization:

```
text = re.sub(r'[^A-Z0-9\s]', '', raw_text.upper()).strip()
```



4.4 EasyOCR Integration

EasyOCR, developed by JaidedAI, employs a deep neural network architecture consisting of a CRAFT (Character Region Awareness for Text detection) detector and a CNN-LSTM recognition network. Unlike Tesseract, which relies on traditional image analysis pipelines, EasyOCR benefits from large-scale training data and neural network generalization. The compare_ocr() function runs both Tesseract (standard and enhanced modes) and EasyOCR on the same plate ROI, printing a side-by-side comparison of their outputs.

4.5 Haar Cascade Detection

As an alternative plate detection method, the system integrates OpenCV's Haar Cascade classifier using a pre-trained XML model (haarcascade_russian_plate_number.xml) downloaded from the official OpenCV GitHub repository. The cascade applies the Viola-Jones detection framework, which uses Haar-like features computed over integral images to identify plate-like rectangular regions. Key detection parameters include a scale factor of 1.1, minimum neighbour count of 5, and a minimum plate size of 80x20 pixels. While computationally efficient, Haar Cascades are less robust to plate orientation and format variations compared to contour-based methods.

4.6 Video Processing Algorithm

The process_video() function extends the single-image pipeline to video inputs through the following mechanism:

- VideoCapture initialization and metadata extraction (FPS, resolution, frame count).
- Frame-by-frame iteration with configurable skip interval (default: every 3rd frame processed).
- For each processed frame: preprocess_image(), detect_plate(), extract_text() are invoked in sequence.
- Detected plate text persists as an overlay across subsequent unprocessed frames to ensure continuous display.
- Frame counter overlay is added for temporal reference.
- Output frames are written to an MP4 video file using VideoWriter with mp4v codec.
- A summary of unique detected plates and sample annotated frames is displayed upon completion.

Sample Annotated Video Frames

Frame #3

Frame #90

Frame #180

5. Results

5.1 Batch Test Results

The system was evaluated on four synthetic Indian number plates using the run_anpr() pipeline without OCR enhancement. Results are summarized below:

#	Plate Text (Expected)	State	Detection Status	OCR Result	Match
1	MH 12 AB 3456	Maharashtra	Plate Detected	MH12AB3456	Partial Match
2	DL 01 CA 7890	Delhi	Plate Detected	DL01CA7890	Partial Match
3	KA 05 MN 2345	Karnataka	Plate Detected	KA05MN2345	Partial Match
4	TN 09 XY 1234	Tamil Nadu	Plate Detected	TN09XY1234	Partial Match

Note: Spaces in the embedded plate text are not always reproduced by Tesseract OCR, as PSM 6 treats the plate as a uniform text block. The whitespace-stripped comparison (detected.replace(' ', '') vs expected.replace(' ', '')) yielded consistent matches across all four test cases, confirming the pipeline's correctness on synthetic inputs.

5.2 Pipeline Stage Analysis

Each stage of the pipeline was evaluated for its contribution to overall performance:

Stage	Key Observation	Performance Note
Grayscale Conversion	Lossless reduction to single channel	Zero computational cost; always succeeds
Bilateral Filter	Effective noise reduction at d=11	Slight processing overhead vs. Gaussian blur; superior edge preservation
Canny Edge Detection	Clearly delineated plate boundaries	Threshold values (30, 200) tuned for synthetic images; may need adjustment for real photos
Contour Detection	Top 30 contours reliably include plate boundary	Aspect ratio filter (1.5-6.5) accommodates all test plates
Otsu Thresholding	Optimal binary separation on white plate background	Excellent on synthetic; may struggle with yellowed or dirty plates

Stage	Key Observation	Performance Note
Tesseract OCR (PSM 6)	Reads alphanumeric characters correctly	Space characters often omitted; regex cleaning required
EasyOCR	Comparable or marginally superior on complex inputs	Higher latency due to neural network inference; better generalization

5.3 Error Handling Validation

The safe_anpr() wrapper was tested against three controlled failure scenarios:

- Non-existent file path: System correctly identifies the missing file and returns an error dictionary without raising an unhandled exception.
- Valid synthetic plate image: Pipeline succeeds end-to-end, returning detected plate text and annotated image.
- Random noise image (200x300, uniform random pixels): No valid plate contour is detected; system returns a descriptive warning with actionable suggestions.

5.4 Comparative OCR Analysis

On the synthetic test plate (MH 12 AB 3456), the compare_ocr() function produced the following comparison:

OCR Engine	Mode	Result	Notes
Tesseract	Standard (PSM 6)	MH12AB3456	Spaces removed; correct characters
Tesseract	Enhanced (Adaptive Threshold)	MH 12 AB 3456	Better spacing with adaptive preprocessing
EasyOCR	Default (GPU=False)	MH 12 AB 3456	Deep learning detection; better spacing

5.5 Performance Metrics

Metric	Value	Condition
Plate Detection Rate (Synthetic)	4/4 (100%)	Clean synthetic images, controlled lighting

Metric	Value	Condition
OCR Character Accuracy (Stripped)	~100%	Whitespace-stripped comparison on synthetic plates
OCR Character Accuracy (With Spaces)	~60-75%	Spaces not always reproduced by Tesseract PSM 6
Video Processing Speed	~3 FPS (CPU)	Google Colab CPU, frame_skip=3, 700x400 resolution
Average Pipeline Latency (per image)	~0.8-1.2 seconds	Including all three stages on Google Colab CPU
Haar Cascade Detection Rate	Variable	Depends on plate angle, lighting, training data

6. Conclusion

6.1 Summary

This project successfully demonstrated the design and implementation of a complete Automatic Number Plate Recognition system using classical computer vision techniques and optical character recognition. The modular pipeline — encompassing bilateral-filtered grayscale preprocessing, Canny-based edge detection, contour-filtered plate localization, Otsu-thresholded OCR enhancement, and Tesseract-based character recognition — achieved reliable detection and extraction of synthetic Indian number plates across four regional formats.

The integration of EasyOCR as a supplementary recognition engine demonstrated the viability of deep-learning-based OCR for number plate applications, with improved handling of spacing and character ambiguities compared to Tesseract. The Haar Cascade detector provided a computationally lightweight alternative detection pathway, though with reduced adaptability to plate format variations. The video processing module extended the system's applicability to real-world traffic surveillance scenarios, with configurable frame-skip optimization balancing detection coverage against computational load.

6.2 Known Limitations

Limitation	Root Cause	Impact
Heavy motion blur	Camera shutter speed too slow for moving vehicles	OCR failure; plate text unrecognizable
Poor nighttime performance	Insufficient illumination for edge detection	Plate not detected; Canny produces noise edges
Steep viewing angle	Perspective distortion of plate geometry	Aspect ratio filter excludes distorted plates
Dirty or damaged plates	Occluded or broken characters	OCR misreads or partially reads plate
Multi-format international plates	Different aspect ratios and character sets	Aspect ratio filter may need country-specific tuning
Reflective plate surfaces	Glare creates saturated regions	Characters masked by overexposure

6.3 Future Enhancements

Several avenues exist for significantly improving system performance and applicability:

- Deep Learning Plate Detection: Replace contour-based detection with YOLOv8 or Faster R-CNN, trained on large-scale annotated plate datasets, for robust performance under real-world conditions.

- Perspective Correction: Implement homographic warp transformation to normalize angled or skewed plate images before OCR.
- Character-level CNN Recognition: Train a custom convolutional neural network on individual character images for direct per-character recognition, eliminating OCR dependency.
- Database Integration: Connect the extracted plate numbers to a vehicle registration database for real-time status lookup (stolen, expired registration, etc.).
- Real-time Streaming: Integrate with WebRTC or RTSP stream protocols for live camera feed processing.
- Multi-country Validation: Implement country-specific regular expression validators to post-process and verify extracted plate strings.
- Night Vision Support: Integrate infrared image preprocessing and contrast enhancement algorithms for nighttime operation.

6.4 Final Remarks

This project affirms that a functional ANPR system can be constructed using freely available, open-source libraries within a cloud-based Python environment, making the technology accessible to a wide audience of researchers, students, and developers. The modular design facilitates straightforward extension and replacement of individual components as more sophisticated techniques become available. The foundation laid by this classical computer vision approach provides a valuable baseline against which deep-learning-based systems can be evaluated and compared.

7. References

7.1 Core Libraries and Tools

- Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools. OpenCV is the primary computer vision library used for image processing, edge detection, and contour analysis in this project.
- Smith, R. (2007). An Overview of the Tesseract OCR Engine. Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR 2007). IEEE. Available at: https://github.com/tesseract-ocr/tesseract
- JaidedAI. (2020). EasyOCR: Ready-to-use OCR with 80+ Languages Supported. GitHub Repository. Available at: https://github.com/JaidedAI/EasyOCR
- Harris, C. R., et al. (2020). Array programming with NumPy. Nature, 585, 357-362. DOI: 10.1038/s41586-020-2649-2
- Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3), 90-95.
- Van der Walt, S., et al. (2014). scikit-image: Image processing in Python. PeerJ, 2, e453.

7.2 Algorithmic References

- Canny, J. (1986). A Computational Approach to Edge Detection. IEEE Transactions on Pattern Analysis and Machine Intelligence, 8(6), 679-698.
- Otsu, N. (1979). A Threshold Selection Method from Gray-Level Histograms. IEEE Transactions on Systems, Man, and Cybernetics, 9(1), 62-66.
- Tomasi, C., & Manduchi, R. (1998). Bilateral Filtering for Gray and Color Images. Proceedings of the 6th International Conference on Computer Vision, 839-846.
- Douglas, D., & Peucker, T. (1973). Algorithms for the Reduction of the Number of Points Required to Represent a Digitized Line or Its Caricature. The Canadian Cartographer, 10(2), 112-122.
- Viola, P., & Jones, M. (2001). Rapid Object Detection using a Boosted Cascade of Simple Features. Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition, 511-518.

7.3 ANPR Domain References

- Anagnostopoulos, C. N. E., et al. (2006). A License Plate Recognition Algorithm for Intelligent Transportation System Applications. IEEE Transactions on Intelligent Transportation Systems, 7(3), 377-392.
- Du, S., Ibrahim, M., Shehata, M., & Badawy, W. (2013). Automatic License Plate Recognition (ALPR): A State-of-the-Art Review. IEEE Transactions on Circuits and Systems for Video Technology, 23(2), 311-325.

- Li, H., Shen, C., & Shi, Q. (2011). Real-Time Visual Tracking Using Compressive Sensing. Proceedings of the 2011 IEEE Conference on Computer Vision and Pattern Recognition, 1305-1312.
- Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 779-788.

7.4 Online Resources

- OpenCV Documentation. Available at: https://docs.opencv.org/ [Accessed 2024].
- Tesseract OCR Documentation. Available at: https://tesseract-ocr.github.io/ [Accessed 2024].
- Google Colab Documentation. Available at: https://colab.research.google.com/ [Accessed 2024].
- OpenCV Haar Cascade Models Repository. Available at: https://github.com/opencv/opencv/tree/master/data/haarcascades [Accessed 2024].
- Python Imaging Library (PIL/Pillow) Documentation. Available at: https://pillow.readthedocs.io/ [Accessed 2024].

Appendix: Key Code Modules Summary

The following table summarizes all major functions implemented in the project notebook:

Cell	Function Name	Purpose	Returns
Cell 3	show_image()	Display single image via Matplotlib	None (side effect: plot)
Cell 3	show_pipeline()	Display multiple pipeline stages side by side	None (side effect: plot)
Cell 3	banner()	Print formatted section separator	None (side effect: print)
Cell 4	preprocess_image()	Grayscale + Bilateral Filter + Canny edges	(gray, blurred, edges)
Cell 5	detect_plate()	Contour-based plate localization	(plate_img, result_img, bbox)
Cell 6	extract_text()	Otsu threshold + Tesseract OCR + clean text	(text, thresh_img)
Cell 7	run_anpr()	Full pipeline orchestrator for single image	(plate_text, result_img)
Cell 8	create_synthetic_plate_image()	Generate synthetic vehicle image with plate	(img, filename)
Cell 11	process_video()	Frame-by-frame video ANPR processing	(detected_plates list)
Cell 13	compare_ocr()	Side-by-side Tesseract vs EasyOCR comparison	None (side effect: print/plot)
Cell 14	detect_plate_haar()	Haar cascade-based plate detection	(plates bounding box list)
Cell 15	safe_anpr()	Error-handled ANPR wrapper	Result dictionary